

Algoritmos II

Árvores binárias de busca

Prof. Rodrigo Minetto

`rminetto@dainf.ct.utfpr.edu.br`

Universidade Tecnológica Federal do Paraná

Motivação: o algoritmo de busca binária, visto nesse curso, tem bom desempenho computacional e deve ser usado quando temos os dados ordenados armazenados em um vetor.

Árvores binárias de busca

Contudo, se precisarmos inserir ou remover elementos e ao mesmo tempo dar suporte a funções de busca eficientes, a estrutura de vetor (e, conseqüentemente, o uso do algoritmo de busca binária) não se mostra adequada.

Árvores binárias de busca

Para inserir um novo elemento em um vetor ordenado temos que reorganizar os elementos no vetor para abrir espaço para a inserção do novo elemento. Uma situação análoga ocorre quando removemos um elemento do vetor.

Árvores binárias de busca

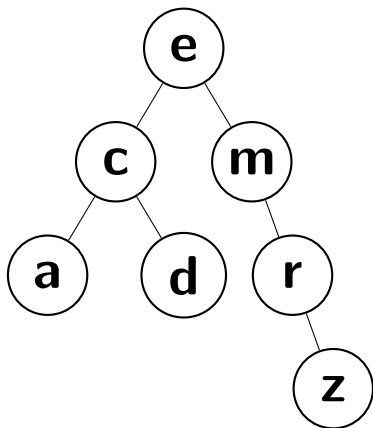
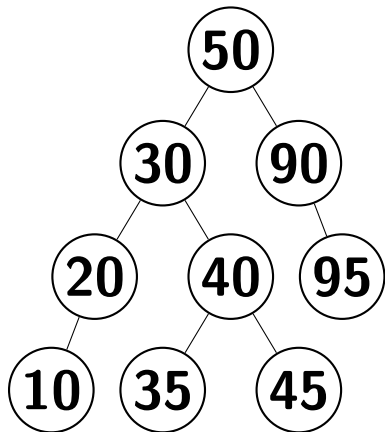
Precisamos de uma estrutura dinâmica que dê suporte a operações eficientes de inserção, remoção e busca!

Solução: **árvores binária de busca!**

Árvores binárias de busca

Árvores binárias de busca têm uma propriedade fundamental: o valor associado à **raiz** é sempre **maior** do que o valor associado a qualquer nó da subárvore à **esquerda** e é sempre **menor** do que o valor associado a qualquer nó da subárvore à **direita**.

Exemplos de árvores binárias de busca



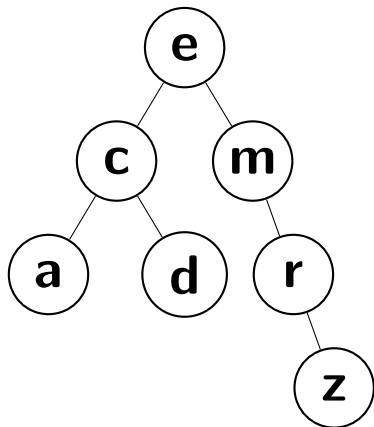
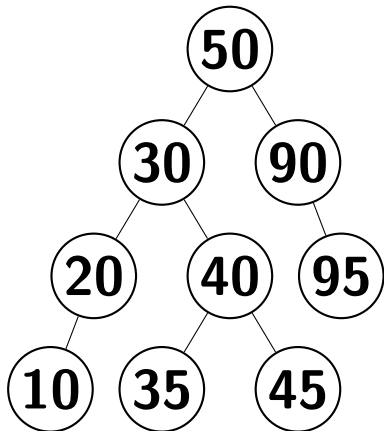
Árvores binárias de busca

Essa propriedade garante que, quando a árvore é percorrida em ordem **simétrica**, ou seja in-ordem (E-R-D), os valores são encontrados em ordem crescente.

Árvores binárias de busca

In-ordem: **10,20,30,35,40,45,50,90,95.**

In-ordem: **a,c,d,e,m,r,z.**



Árvores binárias de busca

Ao usar essa **propriedade de ordem**, a busca de um valor em uma árvore pode ser feita de forma **eficiente**. Para procurar um valor numa árvore, comparamos o valor que buscamos ao valor associado à raiz.

Árvores binárias de busca

Em caso de **igualdade**, o valor foi encontrado; se o valor dado for **menor** que o valor associado à raiz, a busca continua na subárvore **esquerda**; se o valor for **maior** a busca continua na subárvore **direita**.

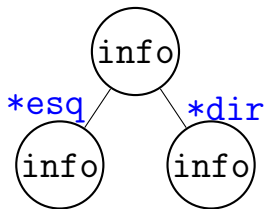
Árvores binárias de busca

Por essa razão, essas árvores são frequentemente chamadas de **árvores binárias de pesquisa**.

Árvores binárias de busca - Estrutura

Tal como anteriormente, uma estrutura em C para representar um **nó da árvore binária de busca** pode ser dada por:

```
typedef struct arvore {  
    int info;  
    struct arvore *esq;  
    struct arvore *dir;  
} Arvore;
```



Árvores binárias de busca - Operações

A operação para **buscar** um elemento explora a propriedade de ordenação da árvore:

```
int buscar (Arvore *a, int v) {  
    if (a == NULL) { return 0; } /*Não achou*/  
    else if (v < a->info) {  
        return buscar (a->esq, v);  
    }  
    else if (v > a->info) {  
        return buscar (a->dir, v);  
    }  
    else { return 1; } /*Achou*/  
}
```

Árvores binárias de busca - Operações

A operação de **inserção** deve adicionar um elemento na árvore na **posição correta** para que a propriedade fundamental da árvore binária de busca seja mantida. Ou seja, comparamos o valor que se deseja inserir com a raiz da árvore e o inserimos na subárvore esquerda ou direita (posição correta localizada recursivamente), conforme o resultado da comparação.

Árvores binárias de busca - Operações

A função a seguir implementa a **inserção**:

```
Arvore* inserir (Arvore *a, int v) {  
    if (a == NULL) {  
        a = (Arvore*)malloc(sizeof(Arvore));  
        a->info = v;  
        a->esq = a->dir = NULL;  
    }  
    else if (v < a->info) {  
        a->esq = inserir (a->esq, v);  
    }  
    else { a->dir = inserir (a->dir, v); }  
    return a;  
}
```


Exercício 1) Desenhe as árvores binárias de busca para os seguintes números inseridos nessa ordem:

- 1, 2, 3, 4, 5, 6, 7.
- 7, 6, 5, 4, 3, 2, 1.
- 4, 6, 2, 5, 1, 7, 3

Qual das três é melhor para buscar um dado elemento.

Exercício 2) Escreva um programa que produza 30 números aleatórios entre 0 e 999. Insira esses números em uma **árvore binária de busca** e imprima os valores na árvore (utilizando um percurso in-order). Os valores estão ordenados?

Exercício 3) Escreva um programa que produza 100000 números aleatórios entre 0 e 99999. Insira esses números em uma **árvore binária de busca**. Procure por um valor que não existe por exemplo 100000. Quanto tempo a busca levou?

Exercício 4) Escreva um programa que produza 100000 em ordem (0 até 99999). Insira esses números em uma **árvore binária de busca**. Procure por um valor que não existe por exemplo 100000. Quanto tempo a busca levou?

Exercício 5) Escreva uma função “min” que encontre uma chave mínima em uma árvore de busca. Escreva uma função “max” que encontre uma chave máxima.